

Layovered — Referral Backend: Jira Tickets

Paste-ready tickets derived from `REFERRAL_BACKEND.md` (+ `REFERRAL_LOOP_SPEC.md`, `REFERRAL_COPY.md`). Epic + 8 tickets, dependency-ordered. Each has scope + acceptance criteria. Estimates are rough.

Conventions: all reward grants are **server-side only**, run inside a **single DB transaction**, and read reward type/value/cap from `referral_config` (never hardcode 5/3/10).

EPIC — LAY-REF: In-app search-credit referral (backend)

Goal: A friend who installs via a referral link, signs in (Google/Apple), and runs their first routing search → referrer gets +5 searches, referee gets +3, both persistent, both auditable, abuse-guarded, and reflected in the searches-remaining API.

Definition of done (epic):

- Referral link → install → auth → first-search → reward works end-to-end on a real device pair.
- Bare installs / anonymous referees / same-device / self-referrals grant nothing.
- Reward value/type/cap come from config; can be changed without a code deploy touching schema.
- `GET /searches/remaining` returns daily + bonus + total; counter UI renders from it.
- K + funnel events fire to analytics.

Ticket order / dependencies: LAY-REF-1 (schema) → LAY-REF-2 (config) → LAY-REF-3 (code on signup) → LAY-REF-4 (/referral/me) → LAY-REF-5 (/referral/attribute) → LAY-REF-6 (activation hook + grant) → LAY-REF-7 (extend /searches/remaining + consume order) → LAY-REF-8 (analytics events). 6 depends on 1-5; 7 depends on 1.

LAY-REF-1 — DB schema: referral tables + users extensions

Type: Task · **Est:** 1–1.5d · **Blocks:** all others

Scope

- Extend `users`: `referral_code` (varchar, unique), `bonus_search_balance` (int, default 0, **persistent — never reset daily**), `auth_provider` (enum google/apple/anonymous), `primary_device_id` (varchar), `referred_by_user_id` (fk users.id, nullable).
- New `referrals` table: `id`, `referrer_user_id` (fk), `referee_user_id` (fk, nullable), `referee_device_id` (varchar), `status` (enum pending/rewarded/rejected), `reject_reason` (varchar nullable: self_referral|same_device|not_authenticated|already_referred|cap_exceeded), `referrer_reward` (json nullable), `referee_reward` (json nullable), `created_at`, `rewarded_at` (nullable).
 - Constraint: **UNIQUE(referee_user_id)** (a user can be referred only once). Index `referrer_user_id`.
- New `search_bonus_ledger`: `id`, `user_id` (fk), `delta` (int), `reason` (enum referral_referrer|referral_referee|search_consumed|manual_adjust), `referral_id` (fk nullable),

`balance_after` (int), `created_at`.

- Device→user mapping for the same-device check: either a `device_user_map(device_id → user_id)` table or reuse `users.primary_device_id` consistently. Pick one and document it.

Acceptance

- Migration runs forward + rollback cleanly on a staging copy.
 - `UNIQUE(referee_user_id)` rejects a second referral row for the same referee.
 - `bonus_search_balance` is untouched by the daily-reset job.
 - `users.bonus_search_balance` always equals the latest `search_bonus_ledger.balance_after` for that user (verified in LAY-REF-6's transaction).
-

LAY-REF-2 — Reward config (single source of truth)

Type: Task · **Est:** 0.5d · **Blocked by:** 1

Scope

- Add `referral_config` (single config row or app-config): `referrer_reward {type:"searches",value:5}`, `referee_reward {type:"searches",value:3}`, `monthly_cap 10`, `activation_event "first_search"`.
- Grant code must **dispatch by type**. Ship the `searches` handler; stub the interface so a `coupon / wallet_credit` handler can be added later **without schema changes**.

Acceptance

- Changing `value 5→7` in config changes the granted amount with **no code change**.
 - Reward snapshots written to `referrals.referrer_reward/referee_reward` reflect config at grant time.
 - An unknown reward `type` fails loudly (logged), never silently grants 0.
-

LAY-REF-3 — Assign referral_code on signup

Type: Task · **Est:** 0.5d · **Blocked by:** 1

Scope

- On user creation, generate a unique short `referral_code` (~8 chars, ambiguity-safe charset).
- Set `auth_provider` and `primary_device_id` at signup/first-auth.
- Backfill `referral_code` for existing users via migration.

Acceptance

- Every new + existing user has a unique `referral_code`.
 - Collision on insert retries and still resolves to unique.
-

LAY-REF-4 — GET /referral/me

Type: Task · **Est:** 0.5d · **Blocked by:** 2, 3

Scope

- Returns: { referralCode, referralLink, bonusBalance, rewardedThisMonth, monthlyCap, rewardConfig }.
- referralLink = ChottuLink base carrying referrerId = referral_code.
- rewardedThisMonth = count of referrals where referrer = me, status = rewarded, rewarded_at in current month.

Acceptance

- Authenticated request returns the caller's code + live balance + monthly count.
- rewardConfig mirrors referral_config (so the app templates copy from it — no hardcoded "+5").
- Unauthenticated request → 401.

LAY-REF-5 — POST /referral/attribute (record pending referral on install)

Type: Task · **Est:** 1d · **Blocked by:** 1, 3

Scope

- Body: { referrerCode, deviceId, refereeUserId? } (refereeUserId null if still anonymous/skip-login).
- Create a referrals row, default status = pending.
- Cheap guards at attribute time (create as rejected w/ reason, don't 500):
 - referrer code doesn't exist → reject already_referred? No — reject as invalid/ignore.
 - referee already has a referral (UNIQUE(referee_user_id)) → already_referred.
 - same device/user as referrer → self_referral / same_device.
- Idempotent: a repeated attribute call for the same install must not create duplicate pending rows.

Acceptance

- Valid call creates exactly one pending row with referee_device_id captured.
- Same-device / self-referral attribution lands as rejected with the right reject_reason.
- Re-posting the same {referrerCode, deviceId} does not duplicate.
- Final anti-abuse validation still happens at activation (this step is best-effort).

LAY-REF-6 — Activation hook + reward grant (the core)

Type: Task · **Est:** 1.5–2d · **Blocked by:** 1, 2, 4, 5 · **Hooks into existing search-consume flow**

Scope

- In the existing routing-search consume path, after a successful search, check: *does this user have a pending referral AND is this their first routing search?*
- If yes, run §4 validation (all must hold):
 - referee auth_provider ∈ {google, apple} (never anonymous/skip-login → leave pending or reject not_authenticated),
 - account genuinely new (created within referral window) + UNIQUE(referee_user_id),

- `referee_device_id` not already mapped to any existing user (esp. the referrer) → else `same_device`,
- `referrer != referee` → else `self_referral`,
- referrer under monthly cap → else `cap_exceeded`.
- On success, in **ONE transaction**:
 - `referrer.bonus_search_balance += referrer_reward.value`; ledger `referral_referrer` with `balance_after`,
 - `referee.bonus_search_balance += referee_reward.value`; ledger `referral_referee` with `balance_after`,
 - `referrals.status = rewarded`, set `rewarded_at`, snapshot both reward jsons,
 - optionally set `referee.referred_by_user_id`.
- On failure → `status = rejected + reject_reason`; grant nothing.
- **Cap policy**: if referrer is over cap, still grant referee's +3, skip referrer's +5; record in snapshot.
- **Idempotent**: status transition + unique constraints prevent double-crediting even on retries/races.

Acceptance

- First-search of a valid referred user → referrer +5, referee +3, both ledgered, status=rewarded, both balances match ledger.
- Second search by same user does **not** re-grant.
- Anonymous referee → stays pending (no grant) until they auth, then grants on their next first-search.
- Same-device, self-referral, over-cap, already-referred → rejected with correct reason, zero grant.
- Concurrent first-searches (race) credit at most once (transaction + constraints proven by test).

LAY-REF-7 — Extend GET /searches/remaining + consume order

Type: Task · Est: 1d · Blocked by: 1

Scope

- Response: `{ "dailyRemaining": int, "bonusBalance": int, "totalRemaining": int }` where `totalRemaining = dailyRemaining + bonusBalance`.
- Consume order on each routing search: decrement `dailyRemaining` **first**; only when daily = 0, decrement `bonusBalance` and write a `search_consumed` ledger entry (`delta = -1, balance_after`).
- Exhausted = both 0 → app shows the "out of searches" wall.

Acceptance

- API returns all three fields; `total = daily + bonus`.
- With `daily>0`, a search reduces `dailyRemaining` and leaves `bonusBalance` untouched (no ledger row).
- With `daily=0` and `bonus>0`, a search reduces `bonusBalance` and writes one `search_consumed` ledger row.
- With both 0, search is refused and the exhausted state is returned.

LAY-REF-8 — Analytics events (K + funnel)

Type: Task · **Est:** 0.5d · **Blocked by:** 5, 6

Scope

- Fire backend/GA4 events: `invite_sent`, `referral_install`, `referral_activated`, `reward_granted` (include searches granted). Enough to compute **viral coefficient K** = invites/user × invite→activation rate, plus the referral→activation funnel and total searches granted (cost).

Acceptance

- Each lifecycle transition emits its event with referral_id + user ids (anonymised as needed).
 - A test referral produces install → activated → reward_granted in analytics.
 - Dashboard/query can derive K and total searches granted (cost) from these events.
-

Suggested sprint split

- **Sprint 1 (foundation):** LAY-REF-1, 2, 3, 7 — schema, config, codes, and the searches API the app counter needs (unblocks the front-end counter immediately).
- **Sprint 2 (the loop):** LAY-REF-4, 5, 6, 8 — endpoints, activation grant, analytics.

LAY-REF-7 is deliberately early: it's what the **search-counter chip + "out of searches" wall** read from, so the app team can build that UI before the full grant logic lands.